Apoorv Agrawal

U.S. Citizen | (309) 706-6931 | apoorv.agrawal240@gmail.com | linkedin.com/in/apoorv240

Education

Bachelor of Science in Computer Engineering (GPA: 3.78/4.00) *Expected May 2029*
Texas A&M University *College Station, TX*

Technical Skills

Programming: C++, C, Python, Java
Embedded Systems: STM32, ESP32, real-time firmware, DMA, UART, CAN, I2C, SPI
Tools: Git, SEGGER Ozone, Altium, KiCad, Fusion 360, Visual Studio
Control & Robotics: Sensor interfacing, motor control, PID / feedforward control
Other: Linux, LaTeX, HTML/CSS/JavaScript, OpenCV, Flask

Technical Experience

Command & Data Handling Engineer | *C++, Linux* *January 2026 – Present*
AggieSat Laboratory – DELOS *College Station, TX*
- Developing real-time embedded software for flight-selected experimental satellite payload running Linux
- using POSIX inter-process communication, implementing shared memory, ring buffers, and mutex/semaphore synchronization to enable data exchange between 5+ processes.
- Writing test plans and procedures for verification and testing of payload software and hardware.

Electrical Engineering Intern | *C++, Altium, STM32, Arduino* *September 2025 – Present*
Aggies Create – Dynamic Survey Tool *College Station, TX*
- Designed and validated a Measure-While-Drilling (MWD) embedded system PCB for subsurface orientation sensing, operating under high vibration and sensor-noise constraints.
- Implementing real-time embedded firmware on STM32 (ARM Cortex-M) with filtering and estimation algorithms (low-pass filters, Extended Kalman Filter) for stable pose estimation.
- Designed and routed high-density PCB traces for STM32 peripherals and external Flash to meet signal-integrity and layout constraints.
- Built a hardware test-fixture prototype by integrating Arduino, stepper motors, and vibration motors to simulate drill behavior.

Undergraduate Researcher & Webmaster | *Python, LaTeX, HTML/CSS/Javascript* *June 2025 – Present*
AGGIES Lab *College Station, TX*
- Building a Python framework to automatically generate standardized data-breach compliance tables for small businesses, substantially cutting manual report prep time for regulatory filings.

Firmware Developer | *C++, SEGGER Ozone* *September 2025 – February 2026*
TAMU Robomasters *College Station, TX*
- Migrated a 10k-LOC C++ motion-control codebase to a breaking library update in a 3-person team by resolving API changes and build issues, restoring stable releases and enabling new features.
- Reduced actuator response latency by **∼30%** by implementing and tuning low-latency feedback and feedforward control loops in embedded C++.

Software Developer | *C++, Python, Visual Studio, ESP32* *August 2025 – January 2026*
Aggie Robotics *College Station, TX*
- Implemented a real-time autonomous path planner to solve collision detection bottlenecks in sparse obstacle fields, achieving sub-50ms trajectory computation through optimized sampling and k-d tree spatial indexing with Informed RRT*.
- Debugged polling- and interrupt-driven 2.4 GHz ESP32–nRF24 radio stack by tuning retry/ACK parameters and analyzing SPI waveforms with a logic analyzer, resolving intermittent packet loss.

Personal Projects

Automated OR Lighting Arm | *ESP32, C++, Python, OpenCV, Flask, HTML/CSS/Javascript* *January 2026*
- Prototyped an IoT operating-room lighting system where a robotic arm tracks a surgeon's pointing hand and automatically repositions the light, demonstrating hands-free surgical lighting control.
- Detected pointing gestures in live video with OpenCV and mapped them to robot joint angles by applying inverse kinematics.
- Implemented a multithreaded Python Flask server for low-latency video streaming, recording/playback, and real-time stability charts, communicating with the ESP32 via a custom HTTP API.
- Quantified hand tremors by building a stability analytics pipeline that applies exponential smoothing over a sliding window.

CPU-Based Software Shader Pipeline | *C++, Multithreading, SIMD, GLSL* *December 2025 – Present*
- Developing a CPU-based 3D software rasterizer in modern C++ targeting 60 FPS at 200k vertices on consumer CPUs by combining SIMD math pipelines and multithreaded work tiling.
- Designing a fully custom GLSL-inspired shader language with a lexer, recursive-descent parser, AST, and bytecode interpreter to enable programmable vertex and fragment stages without GPU APIs.
- Increasing rasterization throughput and CPU utilization by optimizing SIMD math paths, cache-friendly memory layouts, branch behavior, and allocation patterns using only the C++ standard library.

Forza Motorsport Telemetry Display | *Avalonia, C#, Git* *July 2023 – January 2024*
- Implemented a UDP listener and parsing pipeline to decode live Forza telemetry packets and update a desktop UI with minimal latency.
- Designed a responsive MVVM-based Avalonia UI to visualize real-time speed, RPM, gear, and suspension metrics for in-race monitoring.